SIMATS ASSESSMENT

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4303

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool: Application-Based Questions

Weightage: 40%

QUESTIONS: 3 **Total Marks: 30**

1: Online Symposium Portal

A college rv

vis developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
  <input type="text" name="name">
  <input type="email" name="email">
  <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly.

Questions:

1. Identify appropriate **HTML5 semantic elements** missing in the structure.
2. Modify the structure to make it **standards-compliant**.
3. Interpret the **HTTP request generated** during form submission.
4. Analyze the **HTTP response cycle** from server to browser.
5. Suggest improvements for **usability and accessibility**.

2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
  <head>
    <title>Company</title>
  </head>
  <body>
    <div>
      <h1>Welcome
      <p>About us
    </div>
  </body>
</html>
```

Questions:

1. Identify **improper nesting and missing closing tags**.
2. Analyze how different browsers may interpret this structure.
3. Identify **missing semantic elements** (e.g., header, section).
4. Provide a **corrected W3C-compliant HTML structure**.
5. Suggest techniques to ensure **cross-browser compatibility**.

3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="button">Login</button>
</form>
```

Questions:

1. Identify issues in the **form configuration**.
2. Analyze why the **HTTP request is not triggered**.
3. Interpret the role of **GET method in this scenario**.
4. Propose a **corrected implementation**.
5. Suggest improvements for **secure data transmission**.

Code of Conduct:

I Challa Priyadarshini(192311118)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Ch Priyadarshini

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	7.0%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	7.0%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	5.5%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	4.3%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	2.8%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	1.4%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand
Total	**28/30**				

1. Online Symposium Portal

A college is developing an online symposium registration portal. The observed form collects a student's name and email address and submits the data to the /register server endpoint using the HTTP POST method.

Given HTML Structure

```
<form action="/register" method="POST">
   <input type="text" name="name">
   <input type="email" name="email">
   <button>Submit</button>
</form>
```

1.1 Appropriate HTML5 Semantic Elements Missing

The form works syntactically, but it does not provide enough document structure or accessibility information. Appropriate semantic elements include:

- <header> – identifies the introductory area of the symposium portal.
- <main> – identifies the primary content of the page.
- <section> – groups the symposium registration content logically.
- <footer> – identifies footer information such as copyright or contact details.
- <label> – explicitly associates text labels with form controls and improves accessibility.
- <fieldset> and <legend> – useful when a group of related form controls needs a meaningful title.

1.2 Standards-Compliant HTML5 Structure

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Online Symposium Registration</title>
</head>
<body>

<header>
   <h1>College Online Symposium</h1>
   <p>Register to participate in the symposium.</p>
</header>

<main>
   <section aria-labelledby="registration-title">
      <h2 id="registration-title">Student Registration</h2>

      <form action="/register" method="post">
```

```
        <fieldset>
          <legend>Registration Details</legend>

          <label for="name">Full Name:</label>
          <input type="text" id="name" name="name"
                 autocomplete="name" required>

          <label for="email">Email Address:</label>
          <input type="email" id="email" name="email"
                 autocomplete="email" required>

          <button type="submit">Submit Registration</button>
        </fieldset>
      </form>
    </section>
</main>

<footer>
  <p>&copy; 2026 College Symposium</p>
</footer>

</body>
</html>
```

This version uses the HTML5 document declaration, language information, responsive viewport metadata, semantic page regions, explicit labels, unique IDs, required validation, autocomplete hints, and an explicit submit button.

1.3 Interpretation of the HTTP Request

When the student completes the form and activates the Submit button, the browser constructs an HTTP POST request because the form specifies method="post". The action attribute determines the destination endpoint.

```
POST /register HTTP/1.1
Host: www.example.com
Content-Type: application/x-www-form-urlencoded
Content-Length: ...

name=Priyadarshini&email=student%40example.com
```

The exact headers vary according to the browser and server. The important parts are the HTTP method (POST), target path (/register), host, content type, and request body. With the default form encoding, the name and email values are placed in the request body.

1.4 HTTP Response Cycle: Server to Browser

The complete interaction can be understood as a request-response cycle:

- The browser validates the form controls, including required and email-format validation.
- The browser creates the HTTP POST request and sends it to the server.
- The server receives /register and reads the submitted form fields.
- The server validates and processes the registration data, such as checking whether the email is already registered.
- The server creates an HTTP response containing a status code, headers, and optionally an HTML page or redirect.
- The browser receives the response and renders the returned page or follows a redirect.
- For a successful submission, a common design is to return a redirect such as HTTP 303 See Other to a confirmation page, preventing accidental duplicate submissions when the user refreshes.

```
HTTP/1.1 303 See Other
Location: /registration-success

# Browser follows the Location header:
GET /registration-success HTTP/1.1
Host: www.example.com

HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
```

1.5 Usability and Accessibility Improvements

- Use visible <label> elements for every input.
- Use required and appropriate input types such as email.
- Provide clear validation messages when input is missing or invalid.
- Use autocomplete attributes to reduce typing effort.
- Ensure keyboard navigation works without requiring a mouse.
- Maintain sufficient color contrast and visible keyboard focus indicators.
- Use meaningful button text such as 'Submit Registration' instead of an ambiguous label.
- Make the form responsive for mobile devices.
- Associate groups of related controls with <fieldset> and <legend> where appropriate.
- Use accessible error messages and, where necessary, aria-describedby to connect errors with fields.

Expected Result

The improved portal provides a valid HTML5 structure, sends registration data using POST, supports accessible form interaction, and gives users a clear and reliable registration experience.

2. Cross-Browser Compatibility Issue

The company website displays inconsistent layouts because the supplied HTML contains improper nesting and incomplete markup. Browsers use an HTML parser that attempts to repair malformed markup, but different DOM interpretations and browser behaviors can produce unexpected rendering.

Given HTML Structure

```
<html>
<head>
<title>Company</title>
</head>
<body>
<div>
<h1>Welcome
<p>About us
</div>
</body>
</html>
```

2.1 Improper Nesting and Missing Closing Tags

- The <h1> element is not explicitly closed with </h1>.
- The element is not explicitly closed with .
- The content is placed inside a generic without meaningful semantic structure.
- The document does not include <!DOCTYPE html>, so standards mode is not explicitly requested.
- The <html> element should specify the document language, such as lang="en".
- A responsive viewport declaration is missing for reliable mobile rendering.

2.2 How Browsers May Interpret the Structure

Modern browsers use error-recovery algorithms to construct a DOM even when the source HTML is incomplete. For example, a browser may automatically close a paragraph when it encounters another block-level element or when the end of the parent container requires it. However, relying on parser recovery is unsafe. Differences in DOM repair, default CSS styles, quirks mode, and layout engines can make malformed pages behave differently across browsers.

- Some browsers may implicitly insert missing closing tags.
- The final DOM tree may not exactly match what the developer intended.
- Missing <!DOCTYPE html> can trigger quirks mode, changing layout calculations and CSS behavior.
- Default margins and heading/paragraph styles can make spacing appear inconsistent.
- Older browsers or embedded webviews may handle malformed markup differently from current browsers.

2.3 Missing Semantic Elements

- <header> can contain the company name, logo, and introductory content.
- <main> can contain the primary page content.

- <section> can group a meaningful topic such as an About Us section.
- <footer> can contain copyright and contact information.
- <nav> should be used if the page contains site navigation links.

2.4 Corrected W3C-Compliant HTML5 Structure

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Company</title>
</head>
<body>

<header>
   <h1>Welcome to Our Company</h1>
</header>

<main>
   <section aria-labelledby="about-title">
      <h2 id="about-title">About Us</h2>
      <p>We provide reliable technology solutions for our customers.</p>
   </section>
</main>

<footer>
   <p>&copy; 2026 Company. All rights reserved.</p>
</footer>

</body>
</html>
```

2.5 Techniques for Cross-Browser Compatibility

- Always start with <!DOCTYPE html> to enable standards mode.
- Use valid HTML5 structure and close elements correctly.
- Use semantic HTML instead of excessive generic elements.
- Validate markup with the W3C Markup Validation Service during development.
- Use CSS resets or a consistent base stylesheet to reduce browser-default differences.
- Prefer standards-based CSS and avoid browser-specific hacks unless necessary.
- Use feature detection rather than assuming a particular browser.
- Use progressive enhancement so the basic page works even when advanced features are unavailable.
- Test on major browsers such as Chrome, Edge, Firefox, and Safari, including mobile browsers.

- Use responsive CSS techniques such as Flexbox, Grid, and media queries.

Expected Result
The corrected page has a predictable DOM structure, standards mode, semantic elements, valid nesting, and improved portability across modern browsers.

3. Form Submission Failure

The login form contains username and password controls, but its button is explicitly declared as a normal button rather than a submit button. Therefore, activating the button does not submit the form by default.

Given Configuration

```
<form action="/submit" method="GET">
    <input type="text" name="username">
   <input type="password" name="password">
   <button type="button">Login</button>
</form>
```

3.1 Issues in the Form Configuration

- The button uses type="button", which does not submit the form.
- The inputs have no labels, reducing accessibility and usability.
- The fields are not marked required.
- There is no autocomplete guidance such as username and current-password.
- Using GET for login credentials is inappropriate because form values become part of the URL/query string.
- The example does not demonstrate HTTPS, which is essential for protecting credentials in transit.

3.2 Why the HTTP Request Is Not Triggered

The HTML button type is the primary reason. A <button> element inside a form defaults to submit behavior only when its type is omitted or explicitly set to type="submit". Here, type="button" tells the browser that the button is an ordinary interactive button. Unless JavaScript is attached to it to call form submission, the browser does not submit the form and therefore does not send the expected HTTP request.

3.3 Role of GET in This Scenario

GET is designed primarily for retrieving resources. Form controls submitted with GET are encoded into the URL query string. For example, a login submission could look like:

```
GET /submit?username=priya&password=example123 HTTP/1.1
Host: www.example.com
```

This is unsuitable for passwords because URLs can be stored in browser history, logs, analytics systems, proxies, bookmarks, and other infrastructure. GET is also not the appropriate semantic choice for an operation that sends authentication credentials for server-side processing.

3.4 Corrected Implementation

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Secure Login</title>
</head>
<body>

<main>
   <section aria-labelledby="login-title">
      <h1 id="login-title">Login</h1>

      <form action="/submit" method="post">
         <div>
            <label for="username">Username:</label>
            <input type="text"
                  id="username"
                  name="username"
                  autocomplete="username"
                  required>
         </div>

         <div>
            <label for="password">Password:</label>
            <input type="password"
                  id="password"
                  name="password"
                  autocomplete="current-password"
                  required>
         </div>

         <button type="submit">Login</button>
      </form>
   </section>
</main>

</body>
</html>
```

3.5 HTTP Request After Correction

```
POST /submit HTTP/1.1
Host: www.example.com
Content-Type: application/x-www-form-urlencoded

username=priya&password=example123
```

The request body contains the form values rather than putting them into the URL. However, POST alone does not encrypt data. Encryption in transit must be provided by HTTPS/TLS.

3.6 Improvements for Secure Data Transmission

- Use HTTPS/TLS for the entire login page and submission endpoint.
- Use POST rather than GET for password submission.
- Never store plaintext passwords on the server; store strong password hashes using a modern password-hashing algorithm such as Argon2id, bcrypt, or scrypt.
- Use secure, HttpOnly, and appropriately configured SameSite cookies for session management.
- Implement CSRF protection where applicable, especially for cookie-based authentication.
- Validate and sanitize server-side input; do not rely only on HTML validation.
- Apply rate limiting or account lockout controls carefully to reduce credential-stuffing and brute-force attacks.
- Do not place passwords in URLs, logs, error messages, or client-side analytics.
- Use appropriate security headers and keep server-side dependencies updated.
- Return generic authentication failure messages so attackers cannot easily determine whether a username exists.

Expected Result

Changing the button to type="submit" causes the browser to submit the form. Using POST keeps credentials out of the URL, while HTTPS/TLS protects them during network transmission. Additional server-side authentication controls are still required for a secure login system.

4. Overall Comparison and Key Learning Points

Case Study	Main Problem	Correct Approach	Key Concept
Online Symposium Portal	Basic form lacks semantic/accessibility structure	Use HTML5 semantics, labels, validation and POST	HTML5 + HTTP POST + accessibility
Cross-Browser Compatibility	Malformed/incomplete HTML	Use valid nesting, doctype and semantic structure	Standards mode + browser compatibility
Form Submission Failure	Button is type='button' and GET is unsuitable for passwords	Use type='submit', POST and HTTPS	Form submission + web security

5. Conclusion

These three cases demonstrate the importance of writing valid semantic HTML, understanding the HTTP request-response cycle, and designing forms correctly. HTML5 semantic elements improve document structure and accessibility, while standards-compliant markup reduces cross-browser inconsistencies. For forms, the button type determines whether a submission occurs, and the choice between GET and POST must match the purpose and sensitivity of the operation. Authentication forms should use POST over HTTPS and should be supported by secure server-side password and session management.